T3000 API & Protocol

Programs · built Nov 13, 2026

Programs

Control programs for automation, sequences, and custom logic.

Overview

Programs contain custom control logic written in a simplified programming language. They execute continuously to automate building control sequences, implement control strategies, and respond to conditions.

Program Structure

Basic Program

```BASIC
10 REM Zone Temperature Control
20 IF ZONE_TEMP > ZONE_SP + 1 THEN COOLING = ON
30 IF ZONE_TEMP < ZONE_SP - 1 THEN HEATING = ON
40 IF ZONE_TEMP >= ZONE_SP - 0.5 AND ZONE_TEMP <= ZONE_SP + 0.5 THEN GOTO 60
50 GOTO 20
60 COOLING = OFF
70 HEATING = OFF
80 GOTO 20
```

Program Elements

Commands:

- `IF...THEN...ELSE` : Conditional logic
- `GOTO` : Jump to line number
- `GOSUB...RETURN` : Subroutine calls
- `FOR...NEXT` : Loops
- `REM` : Comments

Operators:

- Comparison: `>`, `<`, `>=`, `<=`, `=`, `<>`
- Logical: `AND`, `OR`, `NOT`
- Arithmetic: `+`, `-`, `*`, `/`

Common Applications

Occupancy Control

BASIC

```
REM Office lighting and HVAC control
IF TIME > 0700 AND TIME < 1800 THEN
  OCC_MODE = TRUE
  LIGHTS = ON
  ZONE_SP = 72
ELSE
  OCC_MODE = FALSE
  LIGHTS = OFF
  ZONE_SP = 65
END IF
```

Equipment Sequencing

BASIC

```
REM Chiller staging
IF LOAD > 80% THEN
  IF CH1_STATUS = OFF THEN START_CH1
  WAIT 300  ' 5 minute delay
  IF LOAD > 95% THEN START_CH2
END IF
```

Alarm Logic

```basic
REM High temperature alarm
IF ZONE_TEMP > HI_LIMIT THEN
  ALARM = ON
  SEND_EMAIL("High temp alert")
ELSE
  ALARM = OFF
END IF
```

Program Management

Program Table

View all programs in the Programs page:

#	Label	Status	Last Run	Exec Time	Errors
1	Zone_Control	✅ Running	1s ago	5ms	0
2	Schedule_Mgr	✅ Running	1s ago	2ms	0
3	Alarm_Logic	✅ Running	1s ago	3ms	0

Program Editor

Edit programs directly in the web interface:

1. Click on program row
2. Modify code in editor
3. Click **Validate** to check syntax
4. Click **Save** to store changes
5. Program automatically restarts

Debugging

Runtime Errors

Common errors:

- **Syntax Error**: Invalid command or syntax
- **Division by Zero**: Math error

- **Undefined Variable**: Reference to non-existent point
- **Timeout**: Program execution too long

Troubleshooting

- Use `REM` comments to document logic
- Test in manual mode first
- Monitor execution time
- Check variable values
- Review error logs

Best Practices

1. **Comment Your Code**

```BASIC
REM Author: John Doe
REM Date: 2026-01-06
REM Purpose: Main AHU control
```

2. **Use Meaningful Names**
   - `ZONE_TEMP` not `V1`
   - `OCC_MODE` not `FLAG1`
3. **Error Handling**

```BASIC
IF SENSOR_FAULT = TRUE THEN USE_DEFAULT
```

4. **Execution Time**
   - Keep programs short (<100 lines)
   - Avoid infinite loops
   - Use timers for delays
5. **Version Control**
   - Export programs regularly
   - Document changes
   - Test before deploying

Next Steps

- PID Loops - PID control
- Schedules - Time-based control
- Variables - Program variables

Programs

Program Execution Engine

Script Interpreter

TYPESCRIPT

```typescript
class ProgramInterpreter {
  private variables = new Map<string, number>();
  private labels = new Map<string, number>();
  private callStack: number[] = [];

  async execute(code: string): Promise<void> {
    const lines = code.split('\n');
    let currentLine = 0;

    // Parse labels
    this.parseLabels(lines);

    while (currentLine < lines.length) {
      const line = lines[currentLine].trim();

      if (line.startsWith('REM') || line === '') {
        currentLine++;
        continue;
      }

      const result = await this.executeLine(line);

      if (result.type === 'GOTO') {
        currentLine = result.target;
      } else if (result.type === 'GOSUB') {
        this.callStack.push(currentLine + 1);
        currentLine = result.target;
      } else if (result.type === 'RETURN') {
        currentLine = this.callStack.pop() || lines.length;
      } else {
        currentLine++;
      }
    }
  }
```

```
  private async executeLine(line: string): Promise<ExecutionResult> {
    // Parse and execute line
    if (line.startsWith('IF')) {
      return this.executeIf(line);
    } else if (line.startsWith('GOTO')) {
      return this.executeGoto(line);
    } else if (line.includes('=')) {
      return this.executeAssignment(line);
    }

    return { type: 'CONTINUE' };
  }
}
```

Control Flow

TYPESCRIPT

```typescript
interface ExecutionResult {
  type: 'CONTINUE' | 'GOTO' | 'GOSUB' | 'RETURN';
  target?: number;
}

class ControlFlow {
  executeIf(condition: string, thenClause: string): ExecutionResult {
    const conditionMet = this.evaluateCondition(condition);

    if (conditionMet) {
      if (thenClause.startsWith('GOTO')) {
        const target = parseInt(thenClause.split(' ')[1]);
        return { type: 'GOTO', target };
      } else {
        this.executeLine(thenClause);
      }
    }

    return { type: 'CONTINUE' };
  }

  private evaluateCondition(condition: string): boolean {
    // Parse condition like "TEMP > 72"
    const match = condition.match(/(.+?)\s*([><=!]+)\s*(.+)/);
    if (!match) return false;

    const [, left, op, right] = match;
    const leftVal = this.getValue(left.trim());
    const rightVal = this.getValue(right.trim());

    switch (op) {
      case '>': return leftVal > rightVal;
      case '<': return leftVal < rightVal;
      case '>=': return leftVal >= rightVal;
      case '<=': return leftVal <= rightVal;
      case '=': return leftVal === rightVal;
      case '<>': return leftVal !== rightVal;
      default: return false;
```

```
    }
  }
}
```

Program API

Load and Execute Programs

TYPESCRIPT

```typescript
// Get program list
const programs = await fetch(
  `http://localhost:9103/api/devices/389001/programs`
).then(r => r.json());

// Get program code
const program = await fetch(
  `http://localhost:9103/api/devices/389001/programs/1`
).then(r => r.json());

// Update program
await fetch(`http://localhost:9103/api/devices/389001/programs/1`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    code: `REM Updated program\nIF TEMP > 75 THEN COOLING = ON`,
    enabled: true
  })
});
```

Program Validation

TYPESCRIPT

```typescript
class ProgramValidator {
  validate(code: string): ValidationResult {
    const errors: string[] = [];
    const warnings: string[] = [];
    const lines = code.split('\n');

    for (let i = 0; i < lines.length; i++) {
      const line = lines[i].trim();

      // Check syntax
      if (line.startsWith('IF') && !line.includes('THEN')) {
        errors.push(`Line ${i + 1}: IF without THEN`);
      }

      // Check for undefined variables
      const vars = this.extractVariables(line);
      for (const v of vars) {
        if (!this.isDefined(v)) {
          warnings.push(`Line ${i + 1}: Undefined variable ${v}`);
        }
      }

      // Check goto targets
      if (line.startsWith('GOTO')) {
        const target = parseInt(line.split(' ')[1]);
        if (!this.isValidLabel(target, lines)) {
          errors.push(`Line ${i + 1}: Invalid GOTO target ${target}`);
        }
      }
    }

    return {
      valid: errors.length === 0,
      errors,
      warnings
    };
  }
}
```

Scheduler Integration

Time-Based Execution

TYPESCRIPT

```typescript
class ProgramScheduler {
  private programs = new Map<number, NodeJS.Timeout>();

  schedule(programId: number, interval: number) {
    // Cancel existing schedule
    if (this.programs.has(programId)) {
      clearInterval(this.programs.get(programId)!);
    }

    // Schedule program execution
    const timer = setInterval(async () => {
      try {
        await this.executeProgram(programId);
      } catch (error) {
        console.error(`Program ${programId} error:`, error);
      }
    }, interval);

    this.programs.set(programId, timer);
  }

  async executeProgram(programId: number) {
    const program = await this.loadProgram(programId);
    const interpreter = new ProgramInterpreter();
    await interpreter.execute(program.code);
  }
}
```

Next Steps

- REST API Reference
- WebSocket API
- Variables